DRAFT

Multisig

SECURITY AUDIT REPORT

11 Jul 2026

Prepared by: **Shred Security**

Security Researchers
kenzo - yashar

Table of Contents

About Shred Security

Shred Security provides high quality security audits for blockchain and DeFi protocols across different chains. Our audits consistently uncover high-impact vulnerabilities missed by others, backed by a proven track record of top competition placements and security partnerships with leading protocols.

We also develop open security resources for the broader ecosystem. Our Protocol Deployment Checklist defines baseline deployment-readiness requirements for smart contract protocols, and our Incident Response Checklist provides a structured framework for protocol incident response from alert to safe harbor. HackViz is an interactive platform to learn from, simulate, and visualize past exploits, helping teams build stronger intuition for real-world attack patterns.

Shred Security's mission is to raise the standard of on-chain security through rigorous audits and practical, openly shared security tooling.

Learn more about us: shredsec.xyz

Protocol Executive Summary

Multisig is a minimalist, gas-optimized multisignature wallet protocol for EVM chains. It lets a configurable set of owners collectively authorize transactions through an m-of-n threshold scheme, with authorization proven on-chain via EIP-712 typed ECDSA signatures, pre-registered on-chain approvals, or a sender bypass in which an owner submitting the transaction fills one signature slot with their own `msg.sender`. The wallet additionally implements EIP-1271 (`isValidSignature`) so it can act as a smart-contract signer for external protocols. Owners are stored as a sorted linked list, and every configuration change — adding or removing owners, changing the threshold, delay, or executor, and performing delegatecalls or batched calls — is gated to self-calls, so each change must itself pass through the wallet's own signature flow.

Beyond simple execution, the wallet supports an optional timelock: when a non-zero delay is configured, transactions are queued with an ETA and can only be executed once the delay elapses (or cancelled by the wallet itself). An optional executor address can bypass signature checks or chain execution across nested wallets, enabling hierarchical multisig setups. Wallets are deployed deterministically through the `MultisigFactory` using CREATE2 with minimal-proxy (clone) bytecode. The CREATE2 salt must begin with either the zero address (permissionless / relayer deploys) or the caller's address; only the caller-bound form provides front-running protection, while the zero-prefix path is an intentional open-deploy option that is not front-running protected. The factory's `createWithCalls` path deploys with the factory as a temporary executor to run post-deployment configuration calls that bypass signatures, then hands off to the real executor as the final step. The contract also implements the ERC-721 and ERC-1155 safe-transfer receiver callbacks so wallets can custody NFTs.

Disclaimer

A smart contract security review cannot guarantee the complete elimination of vulnerabilities. The process is limited by time, available resources, and human expertise, and is intended to identify as many potential issues as reasonably possible. As such, no assurance can be given that all vulnerabilities will be discovered, or that the reviewed smart contracts are entirely secure. To strengthen security over time, follow-up audits, bug bounty programs, and continuous on-chain monitoring are strongly recommended.

Risk Classification

Likelihood \ Impact	High	Medium	Low
High	High	High/Medium	Medium
Medium	High/Medium	Medium	Medium/Low
Low	Medium	Medium/Low	Low

Executive Summary

The shred security team has conducted the review for 4 days in total. In this period of time, a total of 4 issues were found: 2 low and 2 informational. No medium- or high-severity issues were identified.

About the Project

Project Name	Multisig Protocol
Repository	https://github.com/z0r0z/multisig
Contracts	Multisig.sol
Commit/PR	2329339
Type of Project	Multisignature Wallet
Lines of Code	~300

Audit Timeline

Audit Start	07/07/2026
Audit End	11/07/2026
Report Published	

Vulnerability Summary

Severity	Count	Fixed	Acknowledged
High Risk	0	0	0
Medium Risk	0	0	0
Low Risk	2	0	0
Informational	2	0	0
Total	4	0	0

Findings Summary

Issue ID	Description	Severity	Status
L-1	Self-referential executor at vanity address causes permanent DoS	Low	Open
L-2	Constructors payable with no ETH recovery	Low	Open
I-1	NotReady(0) conflates "not queued" and "too early"	Informational	Open
I-2	execute re-computes DOMAIN_SEPARATOR on every call	Informational	Open

Findings

Low

[L-1] Self-Referential Executor at Vanity Address Causes Permanent DoS

Severity: Low
Affected Contracts: `Multisig`
Affected Functions: `execute()`, `executeQueued()`, `setExecutor()`

Summary

If a wallet is deployed at a vanity address matching the guard bit pattern (`0x1111` prefix or suffix) and owners later set `executor = address(this)`, every `execute` and `executeQueued` call recurses through the guard hook into itself until the transaction reverts. The wallet becomes permanently unusable.

Technical Details

Guard hooks invoke `Multisig(payable(_executor)).execute(...)` when vanity bits match:

```
if (uint160(_executor) >> 144 == 0x1111)
Multisig(payable(_executor)).execute(target, value, data, sigs);
// ...
if ((uint160(_executor) & 0xFFFF) == 0x1111) {
    Multisig(payable(_executor)).execute(target, value, data,
sigs);
}
```

When `executor == address(this)` and the wallet address satisfies either pattern, each `execute` re-enters itself via the pre- or post-hook. Recovery via `setExecutor` is also blocked, since that path requires a successful `execute`.

Impact

Permanent denial of service for the affected wallet. Funds already deposited remain locked (no theft). Requires deliberate misconfiguration: vanity CREATE2/EIP-7702 address plus a signed `setExecutor(address(this))`. Clone wallets cannot recover; EIP-7702 deployments may recover by revoking delegation.

Recommendation

Reject self-referential executors in `setExecutor`:

```
function setExecutor(address _executor) public payable onlySelf {
    require(_executor != address(this), InvalidConfig());
    emit ChangedExecutor(executor = _executor);
}
```

Alternatively, skip guard hooks when `_executor == address(this)`.

[L-2] Constructors Payable With No ETH Recovery

Severity: Low
Affected Contracts: `Multisig`, `MultisigFactory`
Affected Functions: `constructor()`

Summary

Both `Multisig` and `MultisigFactory` constructors are declared `payable` but neither contract exposes a withdrawal path for ETH sent during deployment.

Technical Details

```
constructor() payable {} // Multisig (implementation)
constructor() payable {} // MultisigFactory
```

The factory implementation is never initialized and holds no owner-controlled recovery function. ETH sent to either constructor address is permanently locked.

Impact

No exploit on normal deployments. Accidental ETH sent to the factory or unreachable implementation contract is unrecoverable. `payable` saves roughly 24 gas at deploy time.

Recommendation

Remove `payable` from constructors if deployment scripts do not require it. Otherwise document that ETH sent to the factory or implementation is permanently locked.

Informational

[I-1] NotReady(0) Conflates "Not Queued" and "Too Early"

Severity: Informational
Affected Contracts: `Multisig`
Affected Functions: `executeQueued()`

Summary

`executeQueued` uses a single `NotReady(eta)` error for both "hash was never queued" and "hash is queued but timelock has not elapsed," making it impossible for callers to distinguish failure modes from revert data alone.

Technical Details

```
uint256 eta = queued[hash];
require(eta != 0 && (msg.sender == address(this) || block.timestamp
>= eta), NotReady(eta));
```

When `queued[hash] == 0` (unknown hash, wrong nonce, or cancelled entry), the revert carries `NotReady(0)`. When the hash is queued but `block.timestamp < eta`, the revert carries `NotReady(futureEta)`. Both cases share the same error selector.

Impact

UX and integrator ergonomics only. Frontends and relayers cannot produce accurate error messages without additional off-chain state lookups. No fund loss or authorization bypass.

Recommendation

Split into distinct errors:

```
error NotQueued();
error NotReady(uint256 eta);

if (eta == 0) revert NotQueued();
if (msg.sender != address(this) && block.timestamp < eta) revert
NotReady(eta);
```

[I-2] `execute` Re-Computes DOMAIN_SEPARATOR on Every Call

Severity: Informational
Affected Contracts: `Multisig`
Affected Functions: `DOMAIN_SEPARATOR()`, `getTransactionHash()`, `execute()`, `isValidSignature()`, `executeQueued()`

Summary

The EIP-712 domain separator is recomputed via `keccak256(abi.encode(...))` on every call rather than cached at initialization, adding unnecessary gas on the hot execution path.

Technical Details

```
function DOMAIN_SEPARATOR() public view returns (bytes32) {
    return keccak256(
        abi.encode(
            keccak256("EIP712Domain(string name,string
version,uint256 chainId,address verifyingContract)"),
            keccak256("Multisig"),
            keccak256("1"),
            block.chainid,
            address(this)
        )
    );
}
```

`getTransactionHash()` calls `DOMAIN_SEPARATOR()` on every `execute` and `executeQueued`. For a static wallet where `chainId` and `address(this)` are fixed, this is redundant work (~500–800 gas per call). OpenZeppelin's `EIP712` caches the separator in immutables; that pattern is not a drop-in fit for CREATE2 clones initialized via `init()`.

Impact

Gas cost only. Correctness is unaffected — the live `block.chainid` read is fork-safe. `view` calls (`eth_call` for signing) do not cost the caller on-chain gas.

Recommendation

Cache the domain separator in `init()` with fork-aware logic (recompute when `block.chainid` changes), or document the tradeoff as an intentional minimal-design.

Appendix

Appendix A: Deployment Checklist Compliance

We compare every protocol's deployment against our published Protocol Deployment Checklist.

Verdict: Partially followed — not fully compliant end-to-end.

The project meets most **design, audit, and on-chain** requirements, but several **deployment-process and operational** checklist items are missing or only partially done. It is **not** safe to call the checklist "fully followed" as written.

What is followed

Pre-deployment (strong)

- **Audits:** Independent audits exist (`audit/report-multisig.md`, `audit/report-mods.md`, etc.) — BLOCKER met.
- **Threat model:** `SECURITY.md` covers deployment risks, trust assumptions, invariants — BLOCKER met.
- **Initialization:** One-shot `init()`, double-init tests, factory-only init — BLOCKER met.
- **CREATE2 / deterministic deploy:** Salts in `vanity_findings.md`, `VanityMiner`, dapp salt mining — BLOCKER met.
- **Multi-chain consistency:** Same addresses across supported chains via SafeSummoner — BLOCKER met.
- **MEV / frontrunning:** Documented (F-1); sender-bound salt mitigation exists — BLOCKER met (with documented tradeoff).
- **Privileged roles:** Owners, executor, EIP-7702 superuser documented — WARNING met.
- **No upgradeable proxy admin:** Correctly N/A for this architecture (immutable clones).

Deployment tooling (adequate for singletons)

- Vanity mining, browser deploy (`script/deploy.html`), dapp wallet deploy flow.
- Documented deployed addresses in `README.md`.
- `create()` / `createWithCalls()` atomically deploy + init.

Testing (strong for unit/integration)

- 280 Foundry tests (201 unit/integration + 33 module + 46 gas benchmarks, plus EIP-7702) covering init, factory, timelock, modules, EIP-7702, and fuzz.

- No CI workflow is present in the repository (`.github/workflows/` is absent); `forge test` is run manually.

What is NOT followed (or only partial)

BLOCKER gaps

Item	Status
Explorer verification on all chains	Not systematically evidenced in-repo (no per-chain verification log)
Bytecode matches audited build	No automated/reproducible verification step documented
Post-deploy on-chain validation	No on-chain validation scripts present in the repo

WARNING gaps (material)

Item	Status
Fork-based mainnet tests	**No** — only `vm.chainId()` simulation, no real fork tests
MEV deployment simulation	**Partial** — documented, not simulated
Deployment txs simulated in Tenderly/Foundry fork	**No** formal fork-based deploy simulation
State verified across multiple RPCs	**Not automated**
Monitoring / alerting for role changes	**Not set up** in repo (operator responsibility)
Secure backups / incident comms	**Not documented** as protocol-owned process

INFO / process gaps

Item	Status
Formal checklist with Yes/No/Evidence per item	Missing; compliance is implicit, not tracked
Deployment runbook	Deploy knowledge scattered across README, dapp, and vanity tooling — no single normative runbook

Item	Status
Emergency drills	Not scheduled
Stale docs (`CancelTx` in mdBook)	Minor hygiene gap

Section-by-section score

Section	Followed?
1.1 Architecture & upgradeability	**Yes** (with correct N/A for non-upgradeable design)
1.2 Threat model & deployment risks	**Mostly** — recovery plan informal; MEV sim missing
1.3 Audits & testing	**Mostly** — fork tests missing
1.4 External assets	**N/A** (token-agnostic core)
2.1 Deployment script sanity	**Mostly** — no single Foundry `Deploy.s.sol`
2.2 Init controls	**Yes**
2.3 Explorer verification	**Partial** — addresses listed, no per-chain evidence trail
2.4 Admin & key management	**Mostly** — operator-side items not enforceable
3.1 Post-deploy sanity	**Partial** — depends on manual `cast` checks
3.2 Smoke tests	**Manual** — not scripted in original repo
3.3 Monitoring	**No**
3.4 Incident response	**Partial** — recovery logic in code/docs, no formal runbook

---

Bottom line

For **already-deployed singleton contracts** (factory, implementation, TimelockExecutor): the **technical and security foundations are solid** — audited, tested, deterministic, correctly initialized.

For the **checklist as a complete deployment discipline**: **not fully followed**. The main gaps are:

1. No fork-based mainnet tests
2. No systematic post-deploy verification trail
3. No monitoring/alerting setup

4. No formal checklist tracking with evidence per item
5. Operational items (backups, incident comms, multi-RPC verification) left to operators

Practical rating: ~70–75% compliant — strong on smart-contract readiness, weak on operational deployment governance.

Appendix B: Stateful Invariant Fuzzing Report

Objective

Complement the existing unit/integration suite with **stateful invariant fuzzing** to stress the core multisig state machine under random sequences of owner management, execution, approvals, and timelock operations.

Methodology

A `MultisigHandler` contract drives random call sequences against a deployed wallet (3 owners, threshold 2, no executor). The fuzzer selects from 12 handler actions per run. After every call, 10 global invariants are checked against on-chain state.

Configuration (`foundry.toml`):

Parameter	Value
`runs`	512
`depth`	64
`fail_on_revert`	`false`

Effective coverage per invariant: ~32,768 handler calls (512 × 64).

Invariants Tested

Mapped to the Key Invariants in `SECURITY.md`:

ID	Invariant	Description
INV-1	Init one-shot	`threshold > 0` at all times
INV-2	Owner linked list	`ownerCount == getOwners().length`; no duplicates; no `SENTINEL` owner
INV-3	Nonce monotonicity	`nonce` never decreases
INV-4	Queued hash latch	Handler-tracked pending hashes remain latched in `queued[hash]`
INV-5	Threshold bounds	`0 < threshold <= ownerCount`
INV-6	Immutable implementation	Factory `implementation` address unchanged

ID	Invariant	Description
INV-7	Storage packing	Packed fields (`delay`, `nonce`, `threshold`, `ownerCount`) internally consistent
INV-8	`isOwner` consistency	Every `getOwners()` entry passes `isOwner()`
INV-9	Cleared latch hygiene	Ghost-cleared hashes have `queued[h] == 0`
INV-10	Balance sanity	Wallet balance stays bounded

Handler Actions

Action	Exercises
`executeTransfer`	Signed ETH transfer (immediate execution)
`fundWallet`	Refill wallet balance
`addOwner`	Self-call via `execute`
`removeOwner`	Self-call with correct `prevOwner`
`setThreshold`	Threshold change within bounds
`setDelay`	Timelock enable/adjust
`approveHash`	On-chain approval by random owner
`executeWithApprovals`	`v=0` approval slots
`queueTransfer`	Timelock queue path
`warpExecuteQueued`	ETA expiry + `executeQueued`
`cancelQueued`	Self-call `cancelQueued`
`executeQueuedViaSelf`	Self-call wrapped `executeQueued`

Results

Metric	Result
Invariant tests	10 / 10 passed
Fuzz runs per invariant	512
Calls per invariant	32,768
Handler reverts (expected)	~3,700–4,500 per invariant (invalid action preconditions)

Metric	Result
Contract bugs found	**0**

All invariants held across the full fuzz campaign. No violations of owner-list integrity, nonce monotonicity, threshold bounds, or queued-hash latch semantics were observed.

Handler note

During setup, `invariant_queuedLatchConsistent` failed once due to stale pending-entry tracking in the handler after nested timelock paths (`setDelay` → `queueTransfer` → `executeQueuedViaSelf` → `warpExecuteQueued`). This was corrected by adding `_prunePending()` to the handler. **This was a test-harness issue, not a contract defect.**

Out of Scope

The following were intentionally excluded from the handler and should not be inferred as tested:

- Invalid or malformed signatures (handler only submits valid signatures)
- Executor bypass and guardian hooks (`0x1111` vanity address pattern)
- EIP-7702 EOA delegation and EOA superuser semantics
- Module contracts (`AllowlistGuard`, `TimelockExecutor`, etc.)
- `MultisigFactory` / `createWithCalls` deployment paths
- Reentrancy via malicious call targets
- Cross-chain or cross-wallet replay

These areas remain covered by the existing unit/integration suite and prior manual review passes.

Conclusion

Stateful invariant fuzzing found **no issues** in the core `Multisig` state machine under the tested operation sequences. The contract maintains its documented invariants across random interleavings of owner management, signed execution, on-chain approvals, and timelock queue/execute/cancel flows.